

YASH BAVADIYA

[+91-9879163975](tel:+91-9879163975) krbavadiya11@gmail.com [Portfolio](#) [LinkedIn](#) [GitHub](#) [LeetCode](#)

Hiring Team
Sanas, Speech Intelligence & Agentic AI Platform
Bengaluru, India

Dear Hiring Team,

I am a second-year Computer Science student at IIT Jodhpur, and I am applying for the Software Engineering Intern role on the SI & Agentic AI Platform team.

I should mention the scheduling issue up front, since the posting says four months onsite in Bengaluru. My college schedule gives me a three-month break in summer 2027, so that is the window where I can actually be in Bengaluru full time. Outside of that I can work remotely. If either of those works for your team, I would love to be considered. If you need the full four months onsite, I understand, and I would be happy to apply again in a later cycle.

The reason I want to work on this platform is that it is a backend with a lot of data moving through it and real customers on the other end. Most of the projects I have enjoyed have been the same kind of thing. I built DaemonDoc, a service that generates documentation when you push to GitHub. It runs on a Redis queue with BullMQ, and it fails over across six provider keys so one bad API response does not kill a job. The part I liked best was a two-step trick to cut cost: before calling the expensive model, I send only the file paths and their token estimates to a cheap one and let it pick which files actually matter, so I never pay to ship the whole repo into context. It has handled 959 runs across 68 repositories for 50 users at an 85 percent success rate. Honestly the failing 15 percent taught me more than the rest of it. Figuring out why a job died when it worked fine yesterday was most of the actual work.

I also built Flock, three npm packages for multiplayer cursors and live presence. It passes messages between server instances over Redis pub/sub, and it uses Redis keyspace notifications to clear out users who disconnect badly, so presence data does not go stale for more than 30 seconds. I wrote 58 tests for it covering throttling, eviction, and message delivery. The part I found interesting was accepting that the state is briefly wrong and then deciding how wrong it is allowed to get, which is not something I had thought about before building it.

On the AI side, I spent a semester doing research under Dr. Pratik Mazumder on an offline RAG pipeline in Python, using FAISS for vector search with quantized Phi-3 and Flan-T5-XL to pull answers out of documents. I was mostly guessing at first about which prompts and retrieval settings were better, so I put together an evaluation setup to score them across eight ROUGE metrics. That is what got it to a 71.77 percent improvement over the baseline. It made me think measuring these systems matters more than tweaking prompts by feel.

The posting mentions clear written communication a couple of times, and that is the part of open source I did not expect to care about. I have had 84 or so contributions merged into projects like Kdenlive, Vitest, Wagtail, and croc, and writing up the issue or explaining the change in the pull request was usually harder than the code. I am currently finishing Google Summer of Code with KDE this month, where I fixed a bug in Kdenlive that had been open for 11 years. The fix ships to around 41,000 Flathub installs a month, but it only got merged because I could explain the problem clearly to maintainers who had already seen the bug reported before.

I am comfortable with Python and TypeScript, and I have worked with APIs, SQL, Redis, and Docker. I have not used Kafka or ClickHouse, and I have not worked with real observability tooling, but those are things I would like to learn, and doing it on scoped tickets with engineers who already know them sounds like a good way to start.

Thank you for reading, and for considering my application.

Sincerely,

Yash Bavadiya

B.Tech Computer Science and Engineering, Indian Institute of Technology Jodhpur